Đại học Quốc gia Thành phố Hồ Chí Minh
Trường Đại học Bách khoa
Khoa Khoa học và Kĩ thuật Máy tính



THỰC HÀNH KIẾN TRÚC MÁY TÍNH
(CO2008)

Báo cáo Lab 2

Branches and procedures
(Lệnh rẽ nhánh và thủ tục)

Giảng viên hướng dẫn: Tôn Huỳnh Long
Nhóm: L01
Sinh viên: Đặng Minh Triết - 2510249

Thành phố Hồ Chí Minh, Tháng 11/2026

Mục lục

1 Bài 1 – Làm tròn về bội của 4 gần nhất **2**
1.1 Đề bài . . . 2
1.2 Ý tưởng . . . 2
1.3 Mã nguồn . . . 3
1.4 Kết quả chạy trên MARS . . . 6

2 Bài 2 – Ước chung lớn nhất và bội chung nhỏ nhất **6**
2.1 Đề bài . . . 6
2.2 Ý tưởng . . . 6
2.3 Mã nguồn . . . 7
2.4 Kết quả chạy trên MARS . . . 9

3 Bài 3 – Phần tử lớn thứ hai trong mảng **10**
3.1 Đề bài . . . 10
3.2 Ý tưởng . . . 10
3.3 Mã nguồn . . . 11
3.4 Kết quả chạy trên MARS . . . 14

4 Bài 4 – Giai thừa (đệ quy) **15**
4.1 Đề bài . . . 15
4.2 Ý tưởng . . . 15
4.3 Mã nguồn . . . 16
4.4 Kết quả chạy trên MARS . . . 17

Tài liệu tham khảo **18**

1 Bài 1 – Làm tròn về bội của 4 gần nhất

File code (Github): https://github.com/triet-dang-hcmut/CO2008_Computer_Architechture_Lab/blob/main/Lab_2/file_asm/ex1_lab2.asm
Video demo (Github): https://github.com/triet-dang-hcmut/CO2008_Computer_Architechture_Lab/blob/main/Lab_2/video/video_ex1_lab2.mp4
Video demo (Google Drive): https://drive.google.com/file/d/1BqJv89JMxPq7HVS_rU_Ef0bEJ0wNKMO-/view?usp=sharing

1.1 Đề bài

Viết một chương trình MIPS với các yêu cầu sau:

1. Khai báo một mảng số nguyên có độ dài 5 với dữ liệu được nhập từ bàn phím.
2. Kiểm tra từng phần tử trong mảng có chia hết cho 4 hay không.
3. Nếu một phần tử không chia hết cho 4, đổi nó thành số chia hết cho 4 gần nó nhất (ưu tiên làm tròn xuống). Ví dụ, nếu số là 35 thì kết quả là 36, nếu số là 34 hoặc 33 thì kết quả là 32.
4. In kết quả ra màn hình.

1.2 Ý tưởng

1. **Nhập mảng** (nhãn `input`): dành mảng `array` 20 byte bằng `.space 20` (5 phần tử, mỗi phần tử 4 byte). `$t0` là biến đếm, `$t1` là con trỏ tới phần tử đang nhập. Mỗi vòng lặp in lời nhắc `"Please input element <i>: "`, đọc số bằng `syscall` 5, lưu vào mảng bằng `sw $v0, 0($t1)`, tăng biến đếm thêm 1 và con trỏ thêm 4 byte. Khi đủ 5 phần tử (`beq $t0, 5, print_oldarray`) thì chuyển sang in mảng ban đầu.
2. **In mảng bằng thủ tục `printarray`:** mảng ban đầu và mảng sau khi làm tròn được in theo cùng một dạng `<lời nhắc>[a0, a1, a2, a3, a4]`, chỉ khác lời nhắc (`"Old array: "` hoặc `"New array: "`). Vì vậy phần in được viết thành một thủ tục dùng chung:
   - Nơi gọi (`print_oldarray`, `print_newarray`) đặt địa chỉ chuỗi lời nhắc vào `$a1` rồi gọi `jal printarray`. Lệnh `jal` lưu địa chỉ lệnh kế tiếp vào `$ra` rồi nhảy tới thủ tục.
   - Thủ tục in lời nhắc (`move $a0, $a1`) và `"["`, sau đó đặt `$t1` = địa chỉ `array[0]`, `$t0` = `array` $+16$ là địa chỉ phần tử cuối `array[4]`.
   - Vòng lặp `loop_printarray` in phần tử tại `$t1`. Nếu `$t1` = `$t0` (vừa in phần tử cuối) thì thoát vòng lặp; nếu chưa thì in `", "` rồi tăng `$t1` thêm 4. Nhờ kiểm tra điều kiện dừng **trước** khi in dấu phẩy nên không bị thừa dấu phẩy ở cuối.
   - Cuối thủ tục (`end_printarray`) in `"]"` và xuống dòng, rồi trở về nơi gọi bằng `jr $ra`. Sau khi in mảng cũ, chương trình nhảy sang `check`; sau khi in mảng mới thì kết thúc.
   - `printarray` là thủ tục lá (không gọi thủ tục nào khác) nên `$ra` không bị ghi đè, không cần cất `$ra` lên stack. Thủ tục có dùng `$t0`, `$t1`, nhưng đây là các thanh ghi tạm và `check` gán lại chúng ngay từ đầu nên không ảnh hưởng.
3. **Kiểm tra chia hết cho 4** (nhãn `check`/`div_4`): đặt `$s0` = 4, với mỗi phần tử x dùng `div $t2, $s0`, lấy **thương** q bằng `mflo $t3` và **số dư** r bằng `mfhi $t4`. Nếu $r = 0$ thì x chia hết cho 4, giữ nguyên và xét phần tử kế tiếp; nếu $r \neq 0$ thì nhảy tới `change`.

4. **Đổi thành bội của 4 gần nhất** (nhãn `change`): kết quả luôn có dạng $k \times 4$, chỉ cần chọn đúng k. Lệnh `div` của MIPS chia làm tròn về 0, nên số dư **cùng dấu với số bị chia**: $x = 4q + r$ với $r \in \{1, 2, 3\}$ khi $x > 0$ và $r \in \{-1, -2, -3\}$ khi $x < 0$.

   - $x > 0$**:** hai bội gần nhất là $4q$ (nhỏ hơn x, cách r) và $4(q+1)$ (lớn hơn x, cách $4 - r$).
     - $r = 1$: $4q$ gần hơn $\Rightarrow k = q$.
     - $r = 2$: cách đều, ưu tiên làm tròn xuống $\Rightarrow k = q$.
     - $r = 3$: $4(q+1)$ gần hơn $\Rightarrow k = q + 1$ (nhãn `round_up_positive`).
   - $x < 0$**:** thương bị làm tròn về 0 nên $4q$ lại là bội **lớn hơn** x (cách $|r|$), còn bội nhỏ hơn là $4(q-1)$ (cách $4 - |r|$).
     - $r = -1$: $4q$ gần hơn $\Rightarrow k = q$.
     - $r = -2$: cách đều, ưu tiên làm tròn xuống (về phía giá trị nhỏ hơn) $\Rightarrow k = q - 1$.
     - $r = -3$: $4(q-1)$ gần hơn $\Rightarrow k = q - 1$.

   Trong code, các trường hợp $r = 1, 2, -1$ nhảy thẳng tới `save`; $r = 3$ nhảy tới `round_up_positive` để tăng `$t3` thêm 1; hai trường hợp còn lại ($r = -2, -3$) giảm `$t3` đi 1. Tại `save`, tính `$t3` $= k \times 4$ bằng `mul`, ghi đè lại vào mảng bằng `sw $t3, 0($t1)` rồi quay lại `div_4` xét phần tử kế tiếp (hoặc sang in mảng mới nếu đã xét xong phần tử cuối).

5. Các lệnh `beq $t4, 1, save`, `bne $t4, 0, change` và `mul $t3, $t3, 4` có toán hạng là hằng số nên là lệnh giả: MARS tự nạp hằng số vào thanh ghi `$at` trước rồi mới thực hiện lệnh với `$at`.

Bảng 1: Các trường hợp làm tròn (thương q từ `mflo`, số dư r từ `mfhi`)

x	q	r	Hai bội gần nhất	k	Kết quả $4k$
35	8	3	32, **36**	$q + 1 = 9$	36
34	8	2	**32**, 36	$q = 8$	32
33	8	1	**32**, 36	$q = 8$	32
32	8	0	–	–	32 (giữ nguyên)
-33	-8	-1	-36, $\mathbf{-32}$	$q = -8$	-32
-34	-8	-2	$\mathbf{-36}$, -32	$q - 1 = -9$	-36
-35	-8	-3	$\mathbf{-36}$, -32	$q - 1 = -9$	-36

1.3 Mã nguồn

```
.data
    array: .space 20
    prompt_input: .asciiz "Please input element "
    colon: .asciiz ": "
    prompt_oldarray: .asciiz "Old array: "
    prompt_newarray: .asciiz "New array: "
    square_bracket_begin: .asciiz "["
    square_bracket_end: .asciiz "]"
    comma: .asciiz ", "
    newline: .asciiz "\n"

```

```
.text
.globl main
main:
    li $t0, 0 # Bien dem
    la $t1, array # array chua 5 so nguyen

# 1. Input
input: # Nhap 5 so nguyen
    # In prompt
    li $v0, 4
    la $a0, prompt_input
    syscall

    li $v0, 1
    move $a0, $t0
    syscall

    li $v0, 4
    la $a0, colon
    syscall

    # Nhap so
    li $v0, 5
    syscall

    # Cap nhat bien dem va array
    sw $v0, 0($t1)
    addi $t0, $t0, 1
    addi $t1, $t1, 4
    beq $t0, 5, print_oldarray
    j input

print_oldarray: # In array cu
    la $a1, prompt_oldarray
    jal printarray
    j check # Chuyen qua check so co chia het cho 4 ko

printarray:
    li $v0, 4
    move $a0, $a1
    syscall

    li $v0, 4
    la $a0, square_bracket_begin
    syscall

    la $t1, array # dia chi array[0]
    la $t0, array
    addi $t0, $t0, 16 # dia chi array[4]

loop_printarray:
    li $v0, 1
    lw $a0, 0($t1)
    syscall

    beq $t1, $t0, end_printarray

    li $v0, 4
    la $a0, comma
    syscall

```

```
    addi $t1, $t1, 4
    j loop_printarray


end_printarray:
    li $v0, 4
    la $a0, square_bracket_end
    syscall

    li $v0, 4
    la $a0, newline
    syscall
    jr $ra # Ve lai print_(old/new)array

# 2. Check
check: # Kiem tra chia het cho 4
    la $t1, array # dia chi array[0]
    la $t0, array
    addi $t0, $t0, 16 # dia chi array[4]
    li $s0, 4

div_4:
    lw $t2, 0($t1)
    div $t2, $s0
    mflo $t3 # Thuong
    mfhi $t4 # So du
    bne $t4, 0, change # Chuyen qua change so ko chia het cho 4
    beq $t1, $t0, print_newarray # Chuyen qua in array moi
    addi $t1, $t1, 4
    j div_4

change: # Chinh so ko chia het cho 4
    # *Xet so duong (so > 0)
    # - Neu du 1 -> thuong * 4 (round down)
    # - Neu du 2 -> thuong * 4 (round down)
    # - Neu du 3 -> (thuong + 1) * 4
    #  *Xet so am (so < 0)
    # - Neu du -1 -> thuong * 4 (round down)
    # - Neu du -2 -> (thuong - 1) * 4
    # - Neu du -3 -> (thuong - 1) * 4
    beq $t4, 1, save
    beq $t4, 2, save
    beq $t4, 3, round_up_positive
    beq $t4, -1, save
    addi $t3, $t3, -1 # Lam tron xuong
    j save

round_up_positive: # Lam tron len
    addi $t3, $t3, 1
    j save

save: # Luu lai so moi
    mul $t3, $t3, 4
    sw $t3, 0($t1)
    beq $t1, $t0, print_newarray # Chuyen qua in array moi
    addi $t1, $t1, 4
    j div_4

print_newarray: # In array moi
    la $a1, prompt_newarray
    jal printarray
```

```
134     j end # Ket thuc chuong trinh
135
136 end:
137     li $v0, 10
138     syscall
```

1.4 Kết quả chạy trên MARS

Chạy với các số trong ví dụ của đề (35, 34, 33) cùng hai số 32 và 10:

```
Please input element 0: 35
Please input element 1: 34
Please input element 2: 33
Please input element 3: 32
Please input element 4: 10
Old array: [35, 34, 33, 32, 10]
New array: [36, 32, 32, 32, 8]
```

Chạy với các số âm và số 0:

```
Please input element 0: -33
Please input element 1: -34
Please input element 2: -35
Please input element 3: -36
Please input element 4: 0
Old array: [-33, -34, -35, -36, 0]
New array: [-32, -36, -36, -36, 0]
```

2 Bài 2 – Ước chung lớn nhất và bội chung nhỏ nhất

File code (Github): https://github.com/triet-dang-hcmut/CO2008_Computer_Architechture_Lab/blob/main/Lab_2/file_asm/ex2_lab2.asm
Video demo (Github): https://github.com/triet-dang-hcmut/CO2008_Computer_Architechture_Lab/blob/main/Lab_2/video/video_ex2_lab2.mp4
Video demo (Google Drive): https://drive.google.com/file/d/1Sm089Q-GfaOSmiXE5hZQzeVOEk_qr_3y/view?usp=sharing

2.1 Đề bài

Viết một chương trình MIPS yêu cầu người dùng nhập hai số nguyên dương a và b (hãy kiểm tra chúng có phải là số dương hay không). Sau đó chương trình in ra ước chung lớn nhất (GCD) và bội chung nhỏ nhất (LCM) của chúng. Các số được nhập sau lời nhắc: `"a = "` và `"b = "`. Ví dụ, nếu người dùng nhập a=6 và b=45 thì kết quả phải là `GCD=3, LCM=90`.

2.2 Ý tưởng

1. **Nhập và kiểm tra a, b:** in lời nhắc `"a = "`, đọc số bằng `syscall` 5. Lệnh `ble $v0, $zero, error` kiểm tra số vừa nhập có ≤ 0 không; nếu có thì nhảy tới `error`, in thông

báo `"Not Positive Value! Please try again..."` rồi kết thúc chương trình. Nếu hợp lệ thì chép sang `$s0`. Số b được nhập và kiểm tra giống hệt, lưu trong `$s1`.

2. **Tính GCD bằng thuật toán Euclid** (nhãn `gcd/loop_gcd`):

$$\gcd(a, b) = \gcd(b,\ a \bmod b), \qquad \gcd(a, 0) = a.$$

   - Chép a, b sang `$t0`, `$t1` để giữ nguyên giá trị gốc trong `$s0`, `$s1` (còn dùng để tính LCM).
   - Mỗi vòng lặp: lấy $r =$ `$t0` $\bmod$ `$t1` bằng `div` và `mfhi`, rồi gán `$t0` $\leftarrow$ `$t1`, `$t1` $\leftarrow r$.
   - Khi `$t1` $= 0$ (`beq $t1, $zero, set_gcd`) thì `$t0` chính là GCD, lưu vào `$s2`.
   - Nếu $a < b$ thì vòng lặp đầu tiên tự đổi chỗ hai số (ví dụ $6 \bmod 45 = 6$ nên cặp $(6, 45)$ thành $(45, 6)$), nên không cần so sánh a, b trước. Số dư luôn giảm dần và ≥ 0 nên vòng lặp chắc chắn dừng.

3. **Tính LCM** theo công thức $\mathrm{LCM}(a, b) = \dfrac{a \times b}{\gcd(a, b)}$. Nếu nhân $a \times b$ trước thì tích có thể vượt quá phạm vi số nguyên 32 bit có dấu ($2^{31} - 1 = 2\,147\,483\,647$), ví dụ $50000 \times 60000 = 3 \times 10^9$, vì `mul` chỉ giữ 32 bit thấp nên kết quả bị sai. Do đó chương trình **chia trước rồi mới nhân**:

$$\mathrm{LCM} = \frac{a}{\gcd(a, b)} \times b.$$

   Vì GCD là ước của a nên phép chia $a / \gcd(a, b)$ luôn chia hết (lấy thương bằng `mflo`), kết quả không thay đổi. LCM được lưu trong `$s3`.

4. **In kết quả** theo dạng `GCD = <GCD>, LCM = <LCM>`.

Bảng 2: Các vòng lặp Euclid khi $a = 6$, $b = 45$

Vòng	`$t0`	`$t1`	`$t0` mod `$t1`	`$t0`, `$t1` mới
1	6	45	6	45, 6
2	45	6	3	6, 3
3	6	3	0	3, 0 $\Rightarrow$ dừng, GCD $= 3$
$\mathrm{LCM} = (6/3) \times 45 = 90$				

2.3 Mã nguồn

```
.data
    prompt_a: .asciiz "a = "
    prompt_b: .asciiz "b = "
    prompt_gcd: .asciiz "GCD = "
    prompt_lcm: .asciiz "LCM = "
    comma: .asciiz ", "
    prompt_negative_number: .asciiz "Not Positive Value! Please try again..."

.text
.globl main
```

```
main: # Doc gia tri a va b
    li $v0, 4
    la $a0, prompt_a
    syscall

    li $v0, 5
    syscall

    ble $v0, $zero, error # a <= 0 -> Loi
    move $s0, $v0 # $s0 = a

    li $v0, 4
    la $a0, prompt_b
    syscall

    li $v0, 5
    syscall

    ble $v0, $zero, error # b <= 0 -> Loi
    move $s1, $v0 # $s1 = b

# Su dung thuat toan Euclid GCD(a,b) = GCD(b,a%b)
gcd:
    move $t0, $s0 # $t0 = a
    move $t1, $s1 # $t1 = b
    j loop_gcd

loop_gcd:
    div $t0, $t1 # a / b
    mfhi $t2 # a % b
    move $t0, $t1 # $t0 = b
    move $t1, $t2 # $t1 = a % b
    beq $t1, $zero, set_gcd # b == 0 -> gcd = a
    j loop_gcd

set_gcd:
    move $s2, $t0 # GCD(a,b)

# Cong thuc LCM(a,b) = (a * b) / GCD(a,b)
lcm:
    move $t0, $s0 # $t0 = a
    move $t1, $s1 # $t1 = b
    div $t0, $s2
    mflo $t0 # a / GCD(a,b)
    mul $s3, $t0, $t1 # (a / GCD(a,b)) * b
    j print

print: # In GCD va LCM
    li $v0, 4
    la $a0, prompt_gcd
    syscall

    li $v0, 1
    move $a0, $s2
    syscall

    li $v0, 4
    la $a0, comma
    syscall

    li $v0, 4
```

```
72      la $a0, prompt_lcm
73      syscall
74
75      li $v0, 1
76      move $a0, $s3
77      syscall
78      j end
79
80  error:
81      li $v0, 4
82      la $a0, prompt_negative_number
83      syscall
84      j end
85
86  end:
87      li $v0, 10
88      syscall
```

2.4 Kết quả chạy trên MARS

Chạy với ví dụ của đề ($a = 6$, $b = 45$):

```
a = 6
b = 45
GCD = 3, LCM = 90
```

Chạy với $a = 50000$, $b = 60000$ ($a \times b$ vượt quá 32 bit nhưng LCM vẫn đúng nhờ chia trước rồi nhân):

```
a = 50000
b = 60000
GCD = 10000, LCM = 300000
```

Nhập số không dương ($a = -3$; và $a = 5$, $b = 0$):

```
a = -3
Not Positive Value! Please try again...

a = 5
b = 0
Not Positive Value! Please try again...
```

3 Bài 3 – Phần tử lớn thứ hai trong mảng

File code (Github): https://github.com/triet-dang-hcmut/CO2008_Computer_Architechture_Lab/blob/main/Lab_2/file_asm/ex3_lab2.asm
Video demo (Github): https://github.com/triet-dang-hcmut/CO2008_Computer_Architechture_Lab/blob/main/Lab_2/video/video_ex3_lab2.mp4
Video demo (Google Drive): https://drive.google.com/file/d/1PfZZQQNDgNWt2v2_mCze-GZVa_w_I_hy/view?usp=sharing

3.1 Đề bài

Viết một chương trình MIPS tìm phần tử lớn thứ hai trong một mảng số nguyên 10 phần tử. Chương trình cũng phải tìm tất cả các chỉ số của nó. In ra giá trị và tất cả các chỉ số của nó. Các phần tử của mảng phải được người dùng nhập sau lời nhắc: `"Please insert a element:"`.
Ví dụ, nếu mảng là 1, 2, 7, -7, 3, 7, 4, 6, 8, 7 thì kết quả phải là `Second largest value is 7, found in index 2, 5, 9.`

3.2 Ý tưởng

Theo ví dụ của đề, giá trị lớn thứ hai là **giá trị lớn nhất trong các phần tử nhỏ hơn giá trị lớn nhất** `max`; các giá trị trùng nhau chỉ tính là một. Ví dụ mảng có hai số 8 là lớn nhất thì giá trị lớn thứ hai vẫn là số nhỏ hơn 8, không phải 8. Nếu mọi phần tử bằng nhau thì không tồn tại giá trị lớn thứ hai.

1. **Nhập mảng** (nhãn `input`): dành 40 byte cho 10 phần tử bằng `.space 40`. `$t0` là con trỏ tới phần tử đang nhập, `$t1` đếm số phần tử đã nhập (bắt đầu từ 1). Mỗi vòng in lời nhắc, đọc số bằng `syscall` 5 và lưu bằng `sw`; nếu `$t1` $= 10$ thì đã nhập đủ, sang in mảng.

2. **In lại mảng** (nhãn `printarray`) theo dạng `Array: [a0, ..., a9]` để dễ kiểm tra kết quả: `$t0` chạy từ `array[0]` tới `$t1` $=$ `array` $+36$ (`array[9]`), chỉ in `", "` khi chưa phải phần tử cuối.

3. **Bước 1 – tìm** `max` (nhãn `max/loop_max`): gán `$s0` $=$ `array[0]` bằng `lw`, rồi duyệt `$t2` từ `array[1]` tới `array[9]`. Mỗi phần tử x được đọc vào `$t3`; nếu `max` $< x$ (`blt $s0, $t3, change_max`) thì gán `max` $= x$.

4. **Bước 2 – tìm giá trị lớn thứ hai và các chỉ số của nó trong một lần duyệt** (nhãn `second_max/loop_second_max`). Các thanh ghi:

   - `$s1`: giá trị lớn thứ hai `second` tìm được tới thời điểm hiện tại.
   - `$t2`: cờ cho biết đã có `second` hay chưa (ban đầu bằng 0). Không gán sẵn `second` $=$ `array[0]` vì `array[0]` có thể chính là `max`.
   - Mảng `index` (`.space 40`) lưu các chỉ số của `second`; `$s3` là con trỏ ghi vào `index`, `$s2` đếm số chỉ số đã lưu.
   - `$t0` trỏ tới `array[i]`, `$t1` là chỉ số i (từ 0 tới 9).

   Với mỗi phần tử x $=$ `array[i]`:

   - x $=$ `max`: bỏ qua.

- Chưa có `second` ($\texttt{\$t2} = 0$) hoặc $x >$ `second`: x là `second` mới (nhãn `new_second`). Gán $\texttt{\$s1} = x$, bật cờ $\texttt{\$t2} = 1$, đồng thời **xóa danh sách chỉ số cũ** bằng cách đặt $\texttt{\$s2} = 0$ và đưa `$s3` về `index[0]`, rồi lưu i.
- $x =$ `second`: lưu thêm i vào `index` (nhãn `update_index`: `sw $t1, 0($s3)`, tăng `$s2` thêm 1 và `$s3` thêm 4).
- $x <$ `second`: bỏ qua.

Vòng lặp dừng khi $\texttt{\$t1} = 9$. Nếu cờ `$t2` vẫn bằng 0 (nhãn `check`) thì mọi phần tử đều bằng `max`, chương trình in `There is no second largest value!`.

5. **In kết quả** (nhãn `printvalue`/`loop_printindex`): in `"Second largest value is "`, giá trị `$s1` và `", found in index "`, rồi in `$s2` phần tử đầu của mảng `index`, cách nhau bởi `", "`. `$t1` đếm số chỉ số đã in, dừng khi $\texttt{\$t1} = \texttt{\$s2}$.

Bảng 3: Bước 2 với ví dụ của đề ($\texttt{max} = 8$)

i	x	Xử lý	`second`	Mảng `index`
0	1	chưa có `second` ⇒ `second` mới	1	0
1	2	$2 > 1 \Rightarrow$ `second` mới, xóa danh sách cũ	2	1
2	7	$7 > 2 \Rightarrow$ `second` mới, xóa danh sách cũ	7	2
3	-7	$-7 < 7 \Rightarrow$ bỏ qua	7	2
4	3	$3 < 7 \Rightarrow$ bỏ qua	7	2
5	7	$7 =$ `second` ⇒ thêm chỉ số	7	2, 5
6	4	$4 < 7 \Rightarrow$ bỏ qua	7	2, 5
7	6	$6 < 7 \Rightarrow$ bỏ qua	7	2, 5
8	8	$8 =$ `max` ⇒ bỏ qua	7	2, 5
9	7	$7 =$ `second` ⇒ thêm chỉ số	7	2, 5, 9

3.3 Mã nguồn

```
.data
    array: .space 40 # Array chua 10 element integer
    index: .space 40
    prompt_input: .asciiz "Please insert an element: "
    prompt_value: .asciiz "Second largest value is "
    prompt_index: .asciiz ", found in index "
    prompt_novalue: .asciiz "There is no second largest value!"
    prompt_array_begin: .asciiz "Array: ["
    prompt_array_end: .asciiz "]\n"
    comma: .asciiz ", "

.text
.globl main
main:
    la $t0, array
    li $t1, 1

input: # Nhap so vao array
    li $v0, 4
```

```
20      la $a0, prompt_input
21      syscall
22
23      li $v0, 5
24      syscall
25
26      sw $v0, 0($t0)
27      beq $t1, 10, printarray
28      addi $t0, $t0, 4
29      addi $t1, $t1, 1
30      j input
31
32  printarray:
33      la $t0, array
34      addi $t1, $t0, 36
35
36      li $v0, 4
37      la $a0, prompt_array_begin
38      syscall
39
40  loop_printarray:
41      li $v0, 1
42      lw $a0, 0($t0)
43      syscall
44
45      beq $t0, $t1, end_printarray
46
47      li $v0, 4
48      la $a0, comma
49      syscall
50
51      addi $t0, $t0, 4
52      j loop_printarray
53
54  end_printarray:
55      li $v0, 4
56      la $a0, prompt_array_end
57      syscall
58
59  max:
60      la $t0, array
61      lw $s0, 0($t0) # $s0 = max = array[0]
62      addi $t1, $t0, 36 # array[9]
63      addi $t2, $t0, 4 # array[1]
64
65  loop_max:
66      lw $t3, 0($t2)
67      blt $s0, $t3, change_max
68
69  update_loop_max:
70      beq $t2, $t1, second_max
71      addi $t2, $t2, 4
72      j loop_max
73
74  change_max:
75      move $s0, $t3
76      j update_loop_max
77
78  second_max:
79      li $s2, 0 # Count index cua second largest
80      la $s3, index # index[0]
```

```
    la $t0, array # array[0]
    li $t1, 0 # Index cua array[i]
    li $t2, 0 # Bool check da co second_max chua

loop_second_max:
    lw $t3, 0($t0)
    beq $t3, $s0, update_loop_second_max # array[i] == max -> bo qua
    beq $t2, $zero, new_second # Chua co second -> Them second moi
    blt $t3, $s1, update_loop_second_max # second moi < second cu -> bo qua
    beq $t3, $s1, update_index # second moi = second cu -> cong index
    blt $s1, $t3, new_second # second moi > second cu -> update second moi

new_second:
    move $s1, $t3
    li $t2, 1 # found = true
    li $s2, 0 # Reset count = 0 trong TH update second moi
    la $s3, index # Ve lai index[0]

update_index:
    sw $t1, 0($s3)
    addi $s2, $s2, 1
    addi $s3, $s3, 4

update_loop_second_max:
    beq $t1, 9, check
    addi $t1, $t1, 1
    addi $t0, $t0, 4
    j loop_second_max

check:
    beq $t2, 0, error

printvalue:
    li $v0, 4
    la $a0, prompt_value
    syscall

    li $v0, 1
    move $a0, $s1
    syscall

    li $v0, 4
    la $a0, prompt_index
    syscall

    la $t0, index
    li $t1, 1

loop_printindex:
    li $v0, 1
    lw $a0, 0($t0)
    syscall

    beq $t1, $s2, end

    li $v0, 4
    la $a0, comma
    syscall

    addi $t1, $t1, 1
    addi $t0, $t0, 4
```

```
142     j loop_printindex
143
144 error:
145     li $v0, 4
146     la $a0, prompt_novalue
147     syscall
148
149 end:
150     li $v0, 10
151     syscall
```

3.4 Kết quả chạy trên MARS

Chạy với ví dụ của đề (mảng 1, 2, 7, -7, 3, 7, 4, 6, 8, 7):

```
Please insert an element: 1
Please insert an element: 2
Please insert an element: 7
Please insert an element: -7
Please insert an element: 3
Please insert an element: 7
Please insert an element: 4
Please insert an element: 6
Please insert an element: 8
Please insert an element: 7
Array: [1, 2, 7, -7, 3, 7, 4, 6, 8, 7]
Second largest value is 7, found in index 2, 5, 9
```

Giá trị lớn nhất xuất hiện nhiều lần (mảng 8, 8, 1, 2, 3, 3, 0, -1, 3, 8):

```
Please insert an element: 8
Please insert an element: 8
Please insert an element: 1
Please insert an element: 2
Please insert an element: 3
Please insert an element: 3
Please insert an element: 0
Please insert an element: -1
Please insert an element: 3
Please insert an element: 8
Array: [8, 8, 1, 2, 3, 3, 0, -1, 3, 8]
Second largest value is 3, found in index 4, 5, 8
```

Mọi phần tử bằng nhau (mảng toàn số 5):

```
Please insert an element: 5
Please insert an element: 5
Please insert an element: 5
Please insert an element: 5
Please insert an element: 5
Please insert an element: 5
Please insert an element: 5
Please insert an element: 5
Please insert an element: 5
Please insert an element: 5
Array: [5, 5, 5, 5, 5, 5, 5, 5, 5, 5]
There is no second largest value!
```

4 Bài 4 – Giai thừa (đệ quy)

File code (Github): https://github.com/triet-dang-hcmut/CO2008_Computer_Architechture_Lab/blob/main/Lab_2/file_asm/ex4_lab2.asm
Video demo (Github): https://github.com/triet-dang-hcmut/CO2008_Computer_Architechture_Lab/blob/main/Lab_2/video/video_ex4_lab2.mp4
Video demo (Google Drive): https://drive.google.com/file/d/1kuFwUclIoVheWIu7g4CctagLTiQv_LiW/view?usp=sharing

4.1 Đề bài

Viết một chương trình MIPS nhận một số nguyên không âm từ người dùng (cần kiểm tra người dùng có nhập đúng số hay không). In giai thừa của số nguyên đó ra màn hình.
Ví dụ, nếu người dùng nhập 5 thì kết quả phải là 120. Nếu người dùng nhập -2 thì kết quả phải là `A factorial for your number can not be found`.

4.2 Ý tưởng

1. **Nhập và kiểm tra n:** đọc n bằng `syscall` 5. Lệnh `blt $v0, 0, error` kiểm tra $n < 0$; nếu đúng thì nhảy tới `error`, in `A factorial for your number can not be found` và kết thúc. Nếu hợp lệ thì chép n vào `$s0`, gọi thủ tục `jal factorial`, rồi `j print` để in kết quả `$s1`.

2. **Thủ tục đệ quy `factorial`** (tham số n trong `$s0`, kết quả trả về trong `$s1`):

$$\text{factorial}(n) = \begin{cases} 1 & \text{nếu } n \leq 1 \\ n \times \text{factorial}(n-1) & \text{nếu } n > 1 \end{cases}$$

   - **Đầu thủ tục:** cấp 2 ô nhớ (8 byte) trên stack bằng `addi $sp, $sp, -8`, cất `$ra` vào `4($sp)` và n (`$s0`) vào `0($sp)`.
   - **Trường hợp cơ sở:** nếu $n \leq 1$ (`bgt $s0, 1, factorial_recursion` không nhảy) thì `$s1` $= 1$ rồi trả về. Trường hợp này bao gồm cả $0! = 1$.

- **Trường hợp đệ quy** (`factorial_recursion`): giảm `$s0` còn $n-1$, gọi `jal factorial` để được `$s1` $= (n-1)!$. Sau khi trở về, lấy lại n của lần gọi này bằng `lw $s0, 0($sp)` rồi tính `$s1` $= (n-1)! \times n$ bằng `mul`.
- **Cuối thủ tục** (`factorial_return`): khôi phục `$ra` từ stack, trả lại 8 byte stack (`addi $sp, $sp, 8`) và trở về nơi gọi bằng `jr $ra`.

3. **Vì sao phải cất cả `$ra` và n lên stack:**
   - Lệnh `jal factorial` bên trong thủ tục ghi đè `$ra`. Nếu không cất, địa chỉ quay về của các lần gọi trước (và cuối cùng là địa chỉ quay về `main`) sẽ bị mất, chương trình không trở về được.
   - Sau khi lời gọi đệ quy trả về vẫn còn phải nhân với n, nhưng trong lúc đệ quy `$s0` đã bị giảm thành $n-1, n-2, \ldots$ Mỗi lần gọi cất n của riêng nó trên stack nên khi trả về lấy lại đúng n để nhân.

Bảng 4: Các lần gọi và trả về khi $n = 5$

Lời gọi	`$s0` (n)	Cất trên stack	Giá trị trả về (`$s1`)
factorial(5)	5	`$ra` (về `main`), 5	$24 \times 5 = 120$
factorial(4)	4	`$ra`, 4	$6 \times 4 = 24$
factorial(3)	3	`$ra`, 3	$2 \times 3 = 6$
factorial(2)	2	`$ra`, 2	$1 \times 2 = 2$
factorial(1)	1	`$ra`, 1	1 (vì $n \leq 1$)

Lưu ý: `mul` chỉ giữ 32 bit thấp của tích nên chương trình cho kết quả đúng với $0 \leq n \leq 12$ ($12! = 479\,001\,600$). Từ $13! = 6\,227\,020\,800$ trở đi kết quả vượt quá phạm vi số nguyên 32 bit có dấu ($2^{31} - 1 = 2\,147\,483\,647$) nên bị sai, ví dụ nhập 13 thì chương trình in ra $1\,932\,053\,504$.

4.3 Mã nguồn

```
.data
    prompt_novalue: .asciiz "A factorial for your number can not be found"

.text
.globl main
main:
    li $v0, 5
    syscall

    blt $v0, 0, error
    move $s0, $v0
    jal factorial
    j print

# factorial (n) = n * factorial(n-1), fact(n) = 1 neu n <= 1
factorial:
    addi $sp, $sp, -8
    sw $ra, 4($sp) # dia chi tro ve
    sw $s0, 0($sp) # n
    bgt $s0, 1, factorial_recursion # n > 1 -> de quy
    li $s1, 1  # n <= 1 -> fact = 1
```

```
22      j factorial_return
23
24  factorial_recursion:
25      addi $s0, $s0, -1 # n-1
26      jal factorial # $s1 = (n-1)!
27      lw $s0, 0($sp) # lay lai n cua lan goi nay
28      mul $s1, $s1, $s0 # n * (n-1)!
29
30  factorial_return:
31      lw $ra, 4($sp) # khoi phuc dia chi tro ve
32      addi $sp, $sp, 8
33      jr $ra
34
35  print:
36      li $v0, 1
37      move $a0, $s1
38      syscall
39      j end
40
41  error:
42      li $v0, 4
43      la $a0, prompt_novalue
44      syscall
45
46  end:
47      li $v0, 10
48      syscall
```

4.4 Kết quả chạy trên MARS

Chạy với hai ví dụ của đề ($n = 5$ và $n = -2$) và trường hợp $n = 0$ (dòng đầu là số được nhập, dòng sau là kết quả):

```
5
120

-2
A factorial for your number can not be found

0
1
```

Tài liệu tham khảo

[1] Pete Sanderson, Kenneth Vollmar. *MARS (MIPS Assembler and Runtime Simulator), phiên bản 4.5.* Truy cập vào 02/11/2026. https://dpetersanderson.github.io/

[2] David A. Patterson, John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface, MIPS Edition.* Morgan Kaufmann.

[3] Khoa Khoa học và Kĩ thuật Máy tính, Trường Đại học Bách khoa – ĐHQG-HCM. *Computer Architecture Lab (CO2008) – Lab 2: Branches and procedures.* 2025.